

CS 5200 HW 1

Solve binary search problems using Divide and Conquer paradigm. You have to write pseudo-code and provide time and space complexity.

Sparse search: Given a sorted array of strings that is interspersed with empty strings, write a method to find the location of a given string

EXAMPLE

Input: ball, {"at", "", "", "", "ball", "", "", "car", "", "", "dad", "", ""}

Output: 4

If the location of the given string is essentially random and/or unpredictable because of the random number of empty strings between words, divide and conquer would not see any practical benefits to linear search. HOWEVER, since the strings are still sorted in place, even with the empty strings, the problem can be significantly sped up when compared to linear search.

I think a basic binary search with a few modifications should work

The logic:

- Standard Binary Search setup (low, high, mid)
- Check A[mid]
- If A[mid] is "", it cannot be compared to the target. It must search outward from mid to find the nearest non-empty string so it can be compared against.
 - If it runs out of bounds before finding a non-empty string, the target isn't there.
 - If it finds a non-empty string, it becomes the new comparison pivot.

- Once the new pivot is a non-empty string, perform a standard Binary Search comparison:
 - Equal? Return index.
 - Target is smaller? Search Left ($high = mid - 1$).
 - Target is bigger? Search Right ($low = mid + 1$).

Pseudocode:

```

FUNCTION SparseSearch(arr, target, low, high)
  IF low > high THEN
    RETURN -1
  END IF

  mid = (low + high) / 2
  tempMid = mid

  // If mid is empty, search both sides to find a non-empty string
  IF arr[mid] == "" THEN
    left = mid - 1
    right = mid + 1

    WHILE true DO
      IF left < low AND right > high THEN
        RETURN -1
      END IF

      // Check right side
      IF right <= high AND arr[right] != "" THEN
        mid = right
        BREAK
      END IF

      // Check left side
      IF left >= low AND arr[left] != "" THEN
        mid = left
        BREAK
      END IF

      right = right + 1
    
```

```

        left = left - 1
    END WHILE
END IF

// Binary search

IF arr[mid] == target THEN
    RETURN mid
ELSE IF arr[mid] > target THEN
    // Target is smaller, look in the left half
    RETURN SparseSearch(arr, target, low, mid - 1)
ELSE
    // Target is larger, look right half
    RETURN SparseSearch(arr, target, mid + 1, high)
END IF
END FUNCTION

```

Complexity:

Time complexity:

Best and average case:

- $O(\log N)$
- If empty strings are scattered, finding a non-empty neighbor takes constant time. The problem size is then cut in half.

Worst case:

- $O(N)$
- If the array consists of one non-empty string at the very end and $N-1$ empty strings, the while loop will scan the entire array just to find the pivot.

Space Complexity:

The iterative implementation would be $O(1)$ since it would be in place but the recursive implementation would be $O(\log N)$ due to the call stack.

Problem 2. Finding local maxima in a rotated array.

Example:

Input: {15,16,19,20,25,1,2,3,4,5,7,10,14}

Output: Maxima could be 25 or 14. Any answer among the two is fine. 20 (I think this is an error. This should be 25) is the maximum, but the question asks for local maxima.

Use a Divide and Conquer approach similar to Binary Search to find this pivot in $O(\log N)$ time.

1. Initialize low and high pointers and calculate mid.
2. Check neighbors:
 - If $\text{arr}[\text{mid}]$ is greater than $\text{arr}[\text{mid} + 1]$, then $\text{arr}[\text{mid}]$ is the peak so return mid.
 - If $\text{arr}[\text{mid}]$ is smaller than $\text{arr}[\text{mid} - 1]$, then $\text{arr}[\text{mid} - 1]$ is the peak so return mid - 1.
3. **Decide direction:**
 - If $\text{arr}[\text{low}] \leq \text{arr}[\text{mid}]$, it means the left half is strictly sorted. Therefore, the break of the array must be in the right half.
 - If $\text{arr}[\text{low}] > \text{arr}[\text{mid}]$, it means the unsorted rotation break occurred somewhere in the left half, so search to the left

Pseudocode:

FUNCTION FindLocalMax(arr, low, high)

```
// Only one element remains
IF low == high THEN
    RETURN arr[low]
END IF
```

```

mid = (low + high) / 2

// Check if mid itself is the pivot
IF mid < high AND arr[mid] > arr[mid + 1] THEN
    RETURN arr[mid]
END IF

// Check if the element before mid is the pivot
IF mid > low AND arr[mid] < arr[mid - 1] THEN
    RETURN arr[mid - 1]
END IF

// Decide which half to recurse into
IF arr[low] <= arr[mid] THEN
    // Left half is sorted, so the pivot/max must be in the right
half
    RETURN FindLocalMax(arr, mid + 1, high)
ELSE
    // Left half is not sorted, implies the pivot is in the left
half
    RETURN FindLocalMax(arr, low, mid - 1)
END IF
END FUNCTION

```

Complexity:

Time Complexity:

- **Recurrence Relation:** In each step, it performs a constant amount of work and then recurses into exactly one subproblem that is half the size of the original.

$$T(n) = T(n/2) + \Theta(1)$$

- **Derivation:**

- Using the Master Theorem:

- $a = 1$

- $b = 2$

- $f(n) = \Theta(1)$

- Calculate critical exponent: $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$

- Since $f(n) = \Theta(1)$ is equal to $n^{\log_b a}$, this is an example of case 2.
- The solution is $T(n) = \Theta(n^{\log_b a} \lg(n)) = \Theta(1 \lg(n)) = \Theta(\lg(n))$

Space Complexity:

- **Recursive Implementation:** $O(\log(n))$
 - It makes a recursive call for each level of the binary search; the recursion stack will grow to the height of the tree, which is $\log_2(n)$
-

Problem 3. Given a very large array of sorted numbers, assume that x is not present in the array. So, write a divide and conquer algorithm to find the two numbers in the array which surround x .

Example:

2, 5, 7, 13, 50, 100, 120, 130, 300, 500, 800, 1000, 1300, 1500, 1700, 2000.

If x is 1799, then the two number surrounding x are 1700 and 2000. In other words 1799 lies between 1700 and 2000.

Logic:

1. Divide: Calculate the mid index.
2. Conquer: Compare $\text{arr}[\text{mid}]$ with x :
 - If $\text{arr}[\text{mid}] < x$, then x must lie in the right half. Recurse on the right subarray.

- If $\text{arr}[\text{mid}] > x$, then x must lie in the left half. Recurse on the left subarray.
- 3. Combine: x is not in the array, the base case occurs when the search space is exhausted. At this point, the pointers have crossed:
 - The high pointer will be at the index of the largest number smaller than x .
 - The low pointer will be at the index of the smallest number larger than x .
 - Return these two values as the result.

Pseudocode:

FUNCTION FindSurrounding(arr, low, high, x)

 // Pointers have crossed

 IF low > high THEN

 RETURN arr[high], arr[low]

 END IF

 mid = (low + high) / 2

 // Since x is not present, only check greater or less than

 IF arr[mid] < x THEN

 // x is in the upper half

 RETURN FindSurrounding(arr, mid + 1, high, x)

 ELSE

 // x is in the lower half

 RETURN FindSurrounding(arr, low, mid - 1, x)

 END IF

END FUNCTION

Complexity:

Time Complexity:

- Recurrence Relation: The algorithm performs a constant amount of work and then recurses on exactly one subproblem that is half the size of the input.

- $T(n) = T(n/2) + \Theta(1)$
- **Derivation**
 - **Using Master's Theorem:**
 - $a = 1$
 - $b = 2$
 - $f(n) = \Theta(1)$
 - Calculate critical exponent: $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$
 - Compare $f(n)$ with $n^{\log_b a}$: $f(n) = \Theta(1)$ matches $n^{\log_b a} = 1$. This corresponds to Case 2 of the Master Theorem
 - Result: $T(n) = \Theta(n^{\log_b a} \lg(n)) = \Theta(1 \lg(n)) = \Theta(\lg(n))$

Space Complexity:

- **Recursive:** $O(\log(n))$
 - The recursion depth corresponds to the height of the recursion tree, which is $\log_2(n)$. The stack must store the variables for each active call.